Triage intelligent et copilote de support technique

Contexte. Une plateforme de support technique reçoit des tickets en texte libre ; l'outil doit classer l'urgence et la catégorie, router vers la bonne équipe, **suggérer une réponse ancrée sur l'historique et la base de connaissance**, et **détecter en amont les incidents émergents** (pics anormaux de tickets sur un même sujet) plutôt que de se contenter d'un traitement ticket par ticket.

Architecture

Composant 1 — Classification, routage et priorisation

- Classification multi-label : catégorie, urgence, + signal de sentiment/frustration client (transformer léger DistilBERT/CamemBERT).
- Score de confiance : sous un seuil → file de révision humaine plutôt que routage automatique risqué (human-in-the-loop).
- Priorisation dynamique combinant urgence + sentiment + charge actuelle de l'équipe cible pour ordonner la file de traitement.

Composant 2 — Génération de réponse augmentée par RAG

- SLM (Phi-3.5-mini ou Gemma-2-2B) + RAG sur une base vectorielle : tickets résolus historiques + articles FAQ/base de connaissance.
- Réponse suggérée accompagnée de ses sources (tickets similaires cités) pour vérification par l'agent support avant envoi.
- Détection des questions récurrentes mal couvertes par la base → suggestion automatique de nouveaux articles FAQ, à valider par un humain.

Composant 3 — Détection d'anomalies et d'incidents

- Clustering non supervisé (embeddings de tickets + k-means/HDBSCAN, ou isolation forest) en quasi temps réel sur le flux entrant.
- Objectif : repérer un pic anormal sur une catégorie/un sujet donné (ex. panne d'un service) avant l'explosion du volume, et alerter l'équipe concernée.
- Réutilise le même monitoring de dérive de vocabulaire, mais ici comme signal métier (incident) et pas seulement comme signal de maintenance du modèle.

Modèles et outils suggérés

Composant	Modèle / approche	Remarque
Classification & routage	DistilBERT/CamemBERT multi-tâche (catégorie + urgence + sentiment)	Entraînable en CPU local
Génération RAG	Phi-3.5-mini ou Gemma-2-2B (LoRA/QLoRA) + sentence-transformers + FAISS/Chroma	GPU T4 (Colab/Kaggle) suffisant

Composant	Modèle / approche	Remarque
Détection d'anomalies	sentence-transformers + clustering (k-means/HDBSCAN) ou isolation forest	Non supervisé, pas de fine-tuning lourd

Défis

- **Dérive de vocabulaire** (existant) : data drift sur les embeddings des tickets entrants avec EvidentlyAI, alerte et déclenchement de ré-étiquetage/ré-entraînement.
- **Boucle de feedback humain** : tickets mal routés ou à faible confiance, signalés a posteriori par les agents support, réinjectés dans le pipeline de ré-entraînement — traçabilité des corrections.
- **Évaluation de la génération RAG** : au-delà de l'accuracy, mesurer le grounding (le passage cité soutient-il la réponse ?), le taux de réponses non ancrées, et le taux d'adoption (envoyée telle quelle / modifiée / rejetée).
- **Évaluation de la détection d'anomalies** : peu de labels d'incidents réels → validation semi-supervisée avec les équipes support, arbitrage taux de faux positifs (fatigue d'alerte) vs. délai de détection.
- **Cohérence des trois flux** : score de confiance, sources RAG et alertes d'anomalies consolidés dans un dashboard unique et lisible pour l'agent support.

Livrables

- Pipeline Airflow : ingestion + entraînement/ré-entraînement classification + ré-indexation périodique de la base vectorielle RAG.
- Dashboard Grafana : taux de mauvais routage, taux d'adoption des réponses suggérées, alertes d'anomalies de volume, latence, coût tokens.
- Rapport de dérive EvidentlyAI hebdomadaire (métriques custom de format/complétude incluses).
- Interface de revue humaine : file de tickets à faible confiance, validation des réponses suggérées, validation des nouvelles entrées FAQ.